

EN530.766 Numerical Methods

Sushrut Kumar's Project Report

December, 2021

1 Introduction

A lot of physical system demand solving a linear system of equations, $AX = B$, where to obtain the solution X , matrix A has to be inverted which is a $\mathcal{O}(N^3)$ operation (N : number of solution points). In majority of system A is a sparse matrix (even banded in few) with most of the enteries being 0. Doing an inversion operation on a matrix with mostly zero elements isn't a smart thing. Hence, other methods have to use which operate only on non-zero elements, one such are iterative methods. The iterative methods can still be $\mathcal{O}(N^2)$ and with over-relaxation can be made $\mathcal{O}(N^{1.5})$. The best we can do is $\mathcal{O}(N)$ operation using one such method called Multigrid Method. The advantage of using this method is that it can reduce low wavenumber errors in the solution quite rapidly than the traditional iterative methods like Gauss-Seidel by solving system on a coarse grid where the low wavenumber error becomes high wavenumber errors and can be reduced quite rapidly using a smoother(iterative method). In this work, my attempt to develop a code for the multigrid method is shown. The multigrid method is then used to solve the system as shown below.

$$\frac{\partial^2 u}{\partial x_i^2} = 0 \quad (1)$$

with boundary conditions $u(x, 0) = \sin(4x)$, $u(x, 2\pi) = 0$, $u(0, y) = \sin(4y)$, and $u(2\pi, y) = 0$ where $0 \leq (x, y) \leq 2\pi$

2 Methods

The code for the method was written using Fortran90 and is shown in the appendix. In this work, the smoother used is Gauss-Seidel with and without over-relaxation. The solution at an iteration can be obtained for a uniform grid in a vertex-centered arrangement using,

$$u_{i,j}^{k+1} = \frac{u_{i+1,j}^k + u_{i,j+1}^k + u_{i-1,j}^{k+1} + u_{i,j-1}^{k+1}}{4} - \frac{\Delta^2 S_{i,j}}{4} \quad (2)$$

Here, $S_{i,j}$ is the source term value at point in grid and Δ is the uniform grid spacing.

The same expression can be used for the discretised laplace equation of u ($\mathcal{L}u = 0$) and the error poisson equation, ($\mathcal{L}\epsilon = r$), where $\mathcal{L}$ is the laplacian operator and r is the residual. Figure 1 shows a typical 3-level V-cycle where ϵ'_2 and r'_2 are the newly calculated error and residual for level 2 and ϵ''_2 is the prolonged error from level 2 to 3.

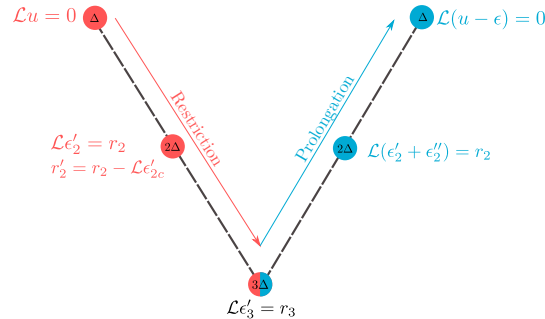


Figure 1: Multigrid V-Cycle

The algorithm is explained in the flowchart (figure 2).

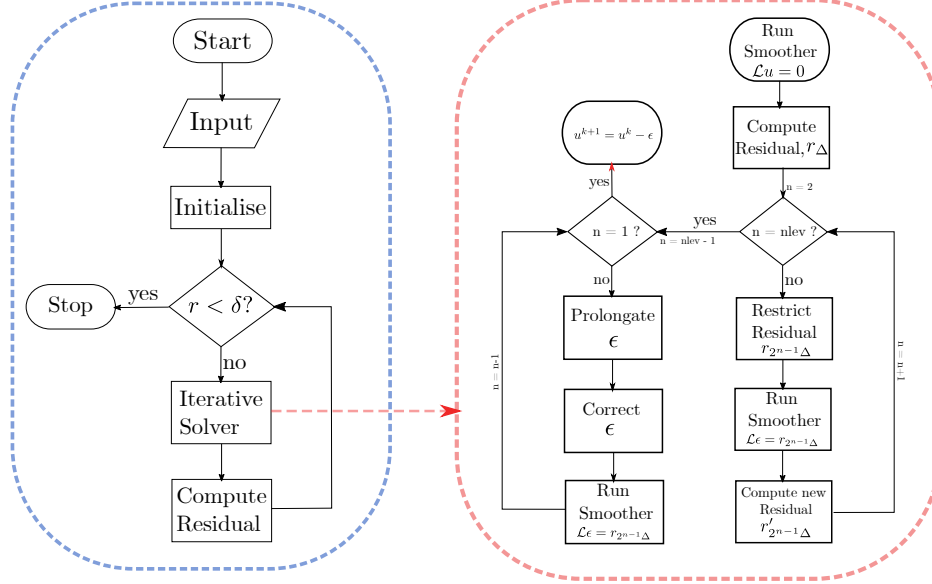


Figure 2: Flowchart of Algorithm (r : Residual and δ : Tolerance)

The first step in the procedure is to initialise the domain with guess value. The choice of guess value is necessary to test the performance of solver for errors in different wavenumber. In this work, I initialised the solution with a random number guess between -1 and 1. While running the simulation for a specific grid resolution the initial guess was **stored** in a file and read again to perform different numerical experiments. After doing one iteration on the **finest level (Level 1)**, the residuals are calculated and restricted to coarse grid of twice the grid spacing. For restriction, an integral preserving weighted averaging was done, where the weights are determined by the ratio of area of fine grid cell in the coarse grid cell. Figure 3a shows the contribution by each of the nodes to the cell center of coarse grid cell. After restriction the error equation is solved with restricted residual as source term and the new residual is calculated. The new calculated residual is again restricted and error equation is solved on a further coarsened grid. The process is repeated again till the lowest level is reached.



Figure 3: Integral Preserving (a) Restriction & (b) Prolongation

After reaching the lowest level in cycle, the error is prolonged (figure 3b) and the existing error at next level is corrected using prolonged error. After correction, the system is again fed to the smoother and new error is obtained. The process is repeated again till the finest level is reached. At the finest level, the error is subtracted from the solution from previous iteration. Then the cycle is repeated again and again till the convergence is reached. For convergence, $\mathcal{L}_2$ norm of residual is considered. After this different parameters were optimised to get the best performance. One thing that greatly speed up computation is **linearly increasing number of smoother iterations between finest and coarsest levels** in restriction. Since performing computation at coarsened levels is cheaper. Similarly number of smoother iterations were linearly decreased in prolongation in intermediate levels.

3 Results

This section presents the results from the simulation. The plots show the contours of u and convergence history of Gauss-Seidel and Multigrid methods for 64×64 (figure 4), 128×128 (figure 5), 256×256 (figure 5), 256×256 (figure 6). The tolerance limit for different grid resolution are kept different but same for two methods. The reason is to have a good comparison as the rate of convergence of Gauss-Seidel is slow and usually takes way longer for higher grid resolution than multigrid to converge. The tolerance chosen is 1×10^{-5} for 256^2 grid, 5×10^{-6} for 128^2 grid & 1×10^{-6} for 64^2 grid. The contours levels are explicitly set to 10 with 0^{th} level excluded from colorbar.

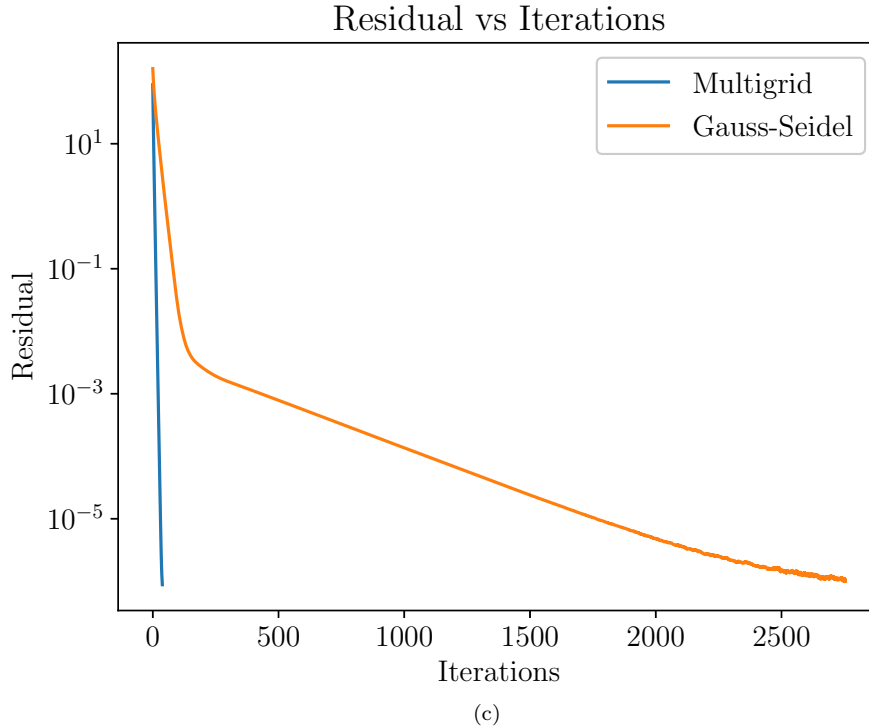
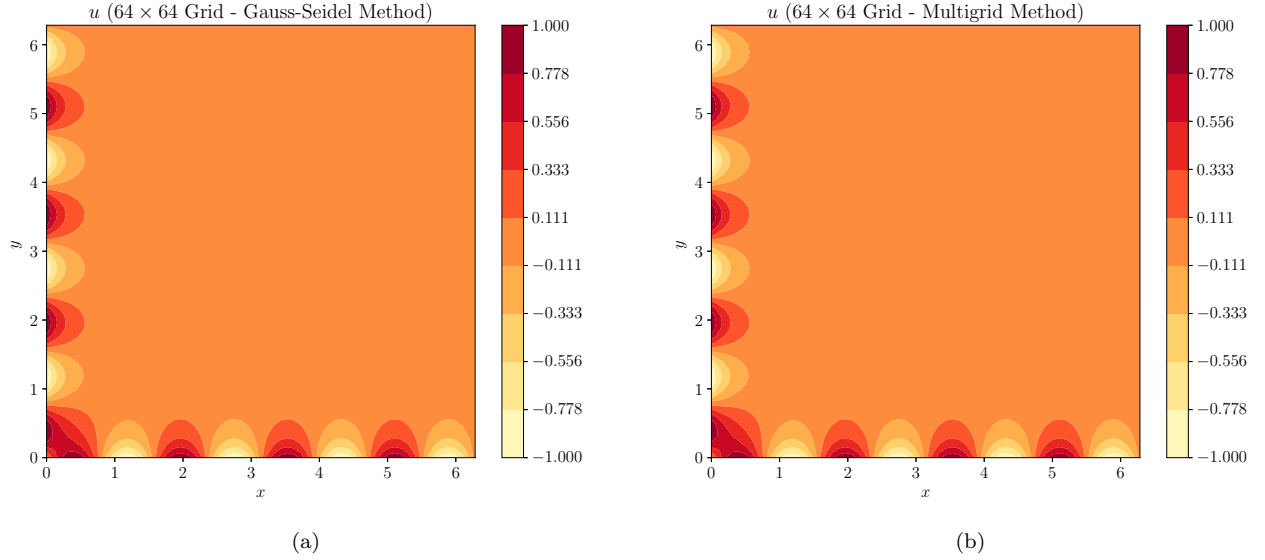


Figure 4: Contour and Convergence history (semilog) for 64×64 grid.

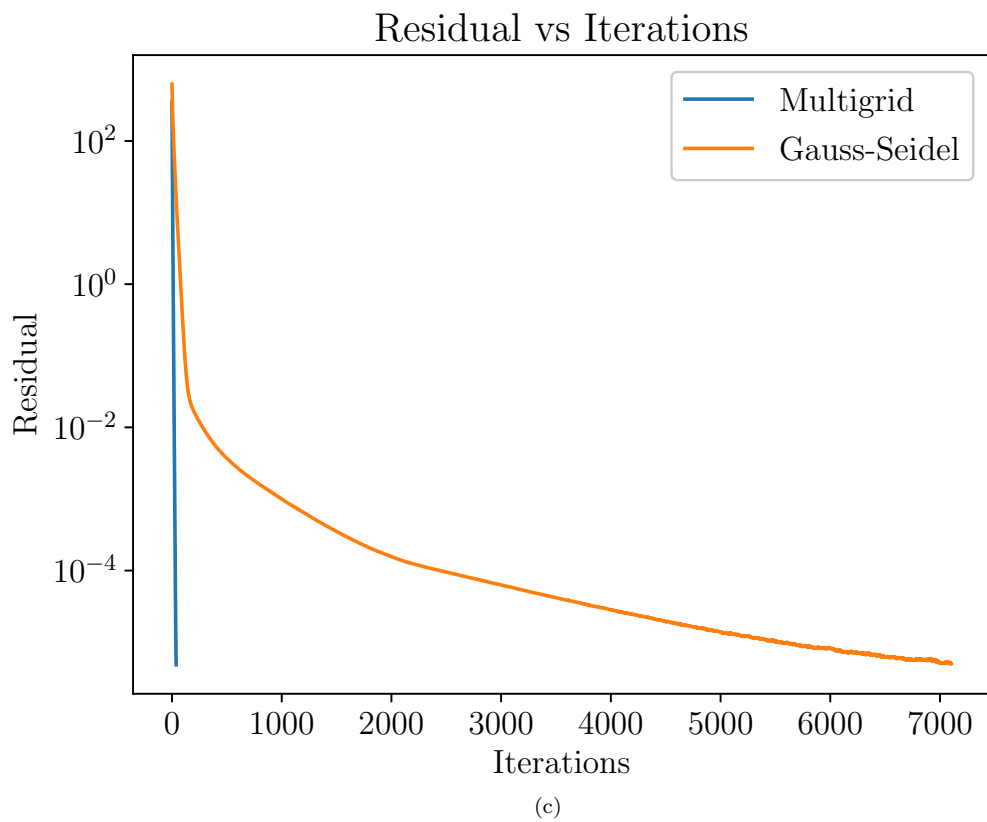
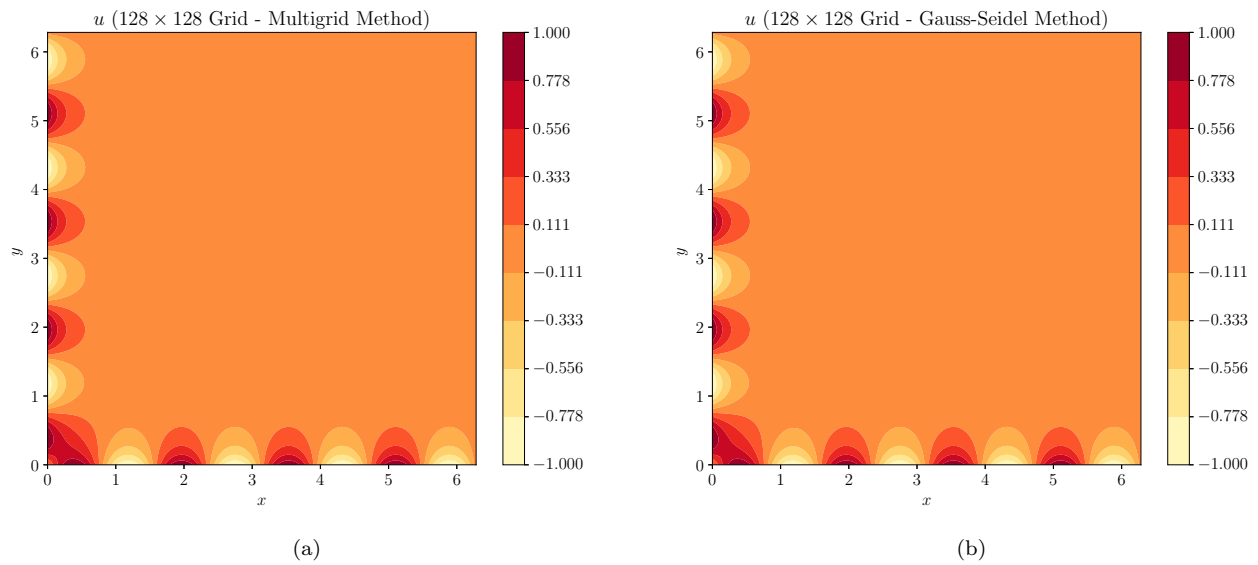


Figure 5: Contour and Convergence history (semilog) for 128×128 grid.

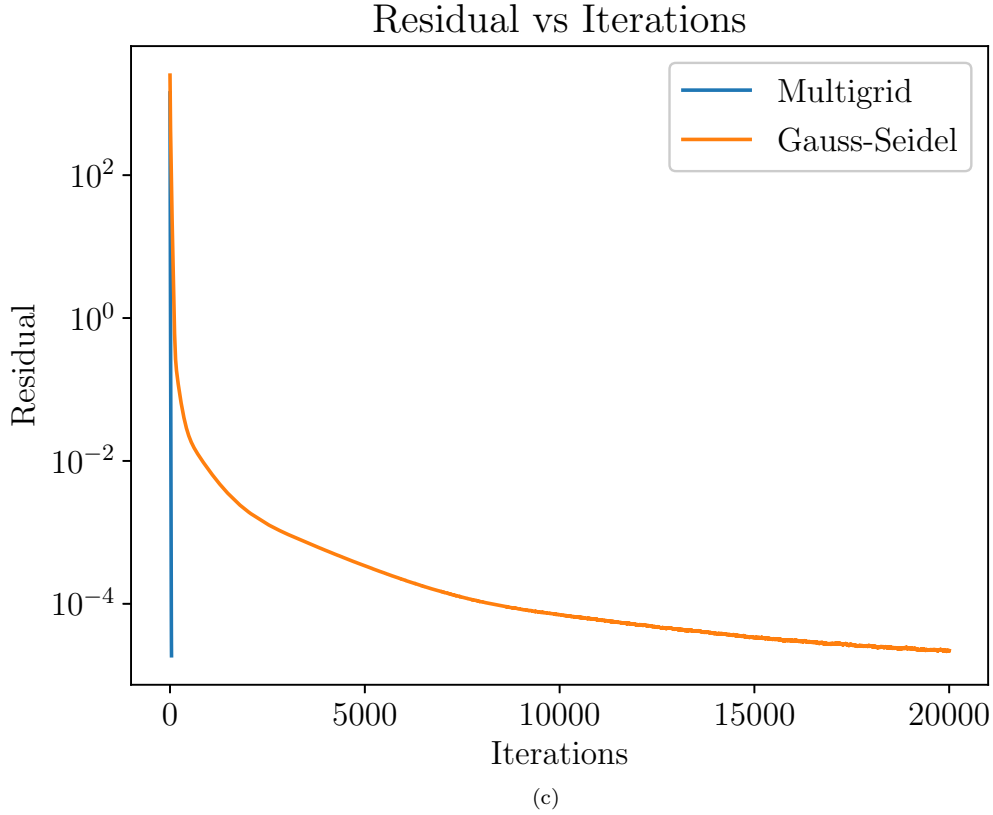
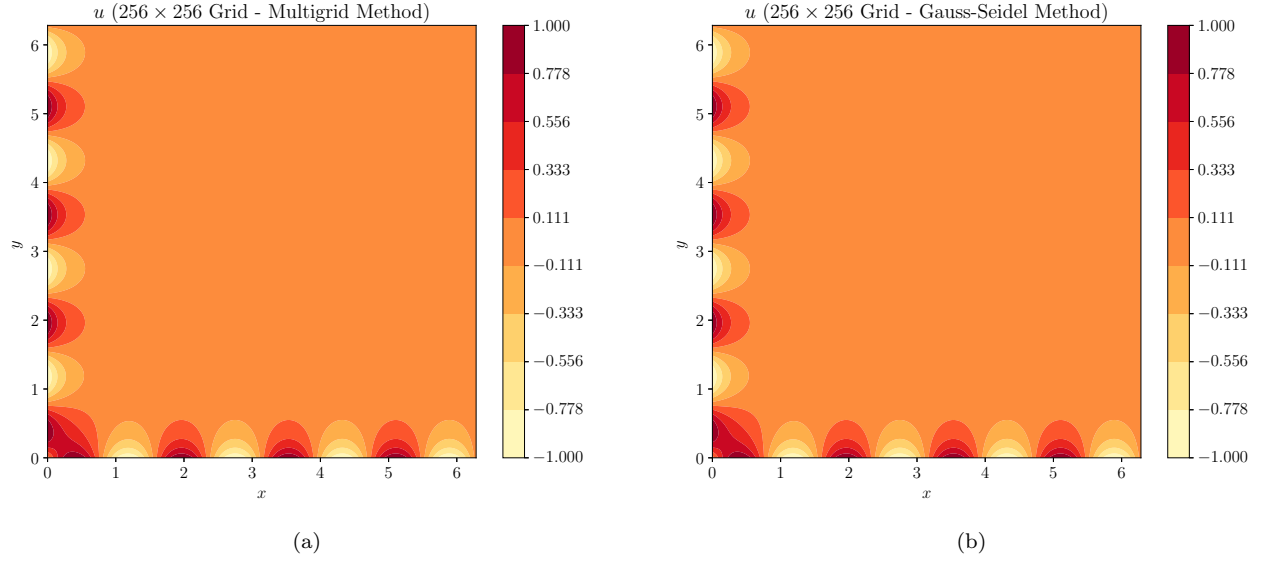


Figure 6: Contour and Convergence history (semilog) for 256×256 grid.

In figure 4, 5, & 6, the contours for both the methods show a diffusion of high wavenumber boundary values into the domain and is in correlation with the physical understanding of the governing equation. Table 1 shows the iterations taken and computation time required by two methods to converge below the specified tolerance limit. It can be seen that at all three grid resolutions Multigrid converged quite rapidly than Gauss-Seidel method. The

trend shown is **similar** to one provided in the **notes**. With increasing grid resolution, the iteration count increase quite rapidly for Gauss-Seidel. Note that the tolerance limit is also increased with grid resolution. At the highest grid (256^2), the Gauss-Seidel didn't converged in the maximum allowable iterations of 20000. Also the results in table is for optimised Gauss-Seidel ($\omega = 1.46$) and Multigrid method.

	Iterations		Computation Time(sec)	
	Gauss-Seidel Method	Multigrid Method	Gauss-Seidel Method	Multigrid Method
64×64 ($\delta = 1 \times 10^{-6}$)	2755	19	4.15×10^{-2}	1.28×10^{-3}
128×128 ($\delta = 5 \times 10^{-6}$)	7103	22	0.5152	4.58×10^{-3}
256×256 ($\delta = 1 \times 10^{-5}$)	20000 (Not Converged)	29	6.44	2.22×10^{-2}

Table 1: Performance comparison of different methods for different grid resolutions (δ -tolerance limit)

4 Discussions

4.1 Effect on convergence due to different levels

Figure 7 show the convergence history for using different coarse grid levels in the Multigrid method for 256^2 grid. The rate of convergence improved drastically by increasing the number of levels in one cycle. The reason behind is the rapid reduction of low wavenumber errors as we go down in levels and decrease the grid resolution where the low wavenumber error act as high wavenumber one and are damped by smoother quite rapidly. Even by using 3 levels the convergence was still better than the Gauss-Seidel.

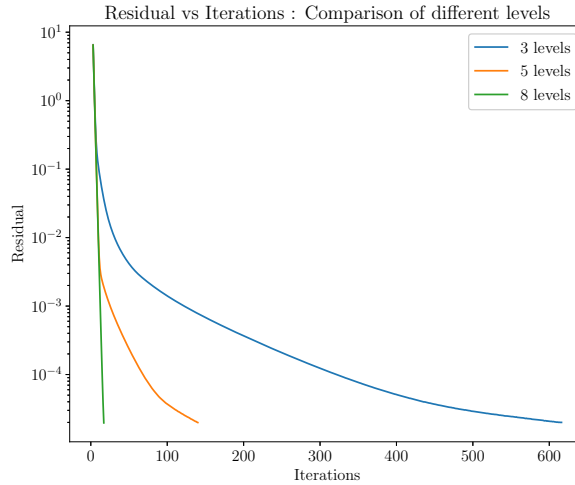


Figure 7: Convergence behaviors of different levels

4.2 CPU time requirement of Mutigrid Method

Table 1 show the time taken by different methods for different grid resolution. In all three cases Multigrid performed better than Gauss-Seidel. But instead of looking at total time required it will be better to look at time required per iteration to see the computational demand of each method.

	Gauss-Seidel Method	Multigrid Method
64×64	9.09×10^{-5}	1.02×10^{-4}
128×128	1.75×10^{-4}	4.23×10^{-4}
256×256	4.47×10^{-4}	1.66×10^{-3}

Table 2: CPU time(sec) required for 1 iteration

Table 2 shows the per iteration time taken by each of these methods for different grid resolutions. Here Gauss-Seidel wins as less work is done in one iteration compared to Multigrid where other task like solving error equation, restriction and prolongation happens.

4.3 Effect of Over-relaxation

Figure 8b shows the amplification factor curve for Gauss-Seidel with and without relaxation. It can be seen that $\omega = 1$ is a better smoother than $\omega > 1$. This is something I also encountered in my simulation. Iteration required with non-relaxed smoother was less than overrelaxed smoother as shown in figure 8a (for 256^2 grid). Although sometimes while running with different random guess values I saw a slightly over-relaxed smoother converges in slightly less iterations but in majority of cases $\omega = 1.0$ showed the best performance.

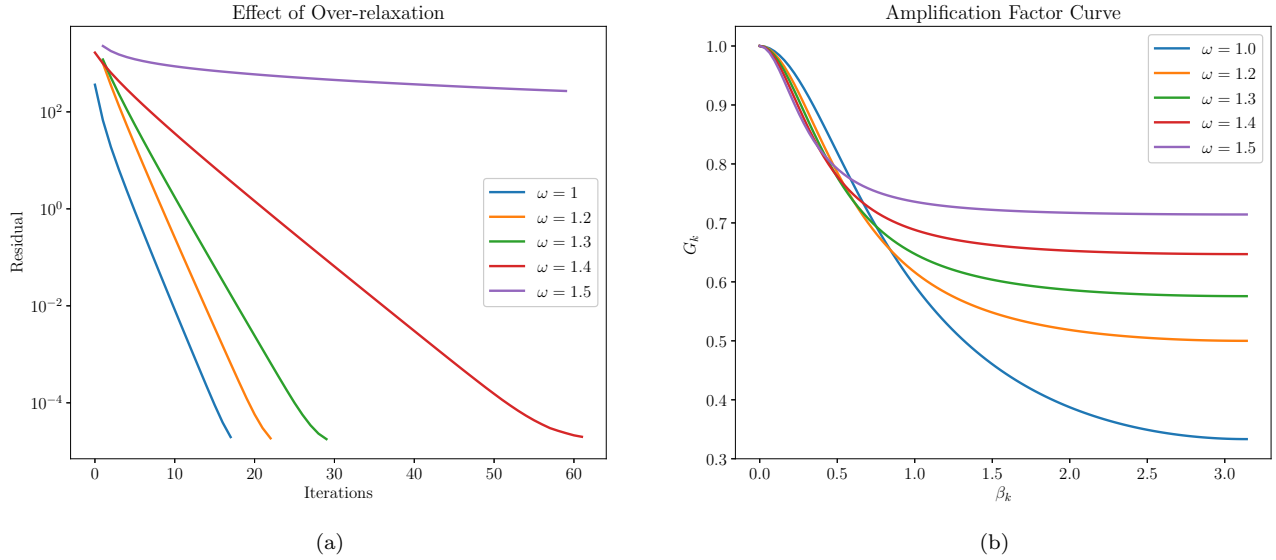


Figure 8: Effect of Over-relaxation, (a) Convergence History, (b) Amplification Factor Curve

4.4 Comparison of different cycles

Figure 9 shows the structure of different cycle followed. This structure was the optimised after running few tests. In W-cycle, after reaching the coarsest level, prolongation is done for 2 levels followed by a restriction to coarsest level and then final prolongation to finest grid level.

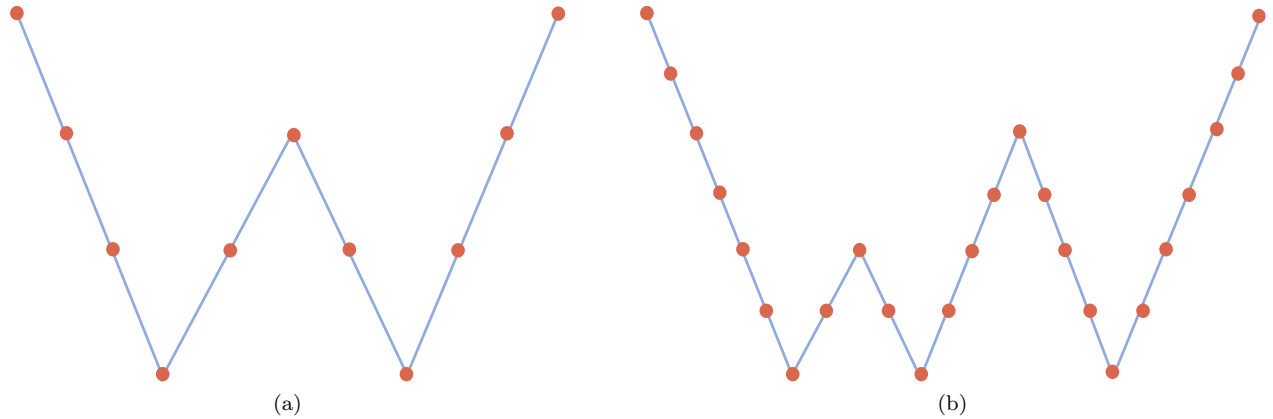


Figure 9: Cycle Structures (a) W-Cycle (b) F-Cycle

Whereas in F-Cycle, after the initial restriction the structure is similar to W-cycle but the second prolongation is not done till the finest level but is stopped 2 levels after the first prolongation level. Figure 10 shows the convergence history for the different cycles used in the Multigrid algorithm and table 3 shows the Iterations required to converge along with the time taken. These tests were done using data from 256^2 grid. In my case, there wasn't much difference between the performance of the 3 cycles. This could be attributed to the level to which the cycles are optimised. In this complex cycle there are various parameters like number of levels, level to which intermediate restriction/prolongation can be done, number of intermediate prolongations/restrictions etc. All these parameters are played with and the most optimum one is used but there is still a potential for further optimisation.

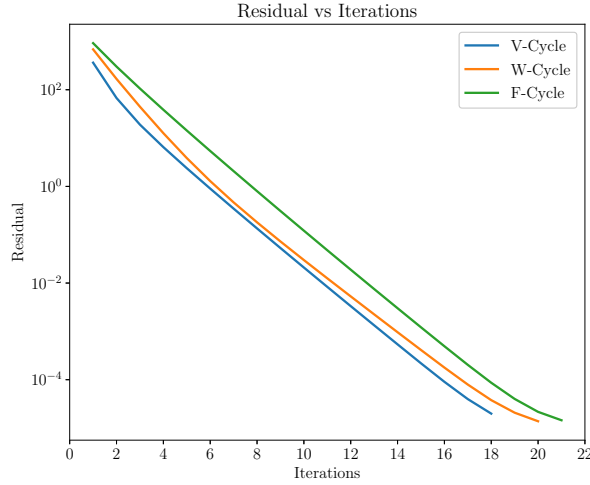


Figure 10: Convergence behaviors of different cycles

	Iterations	CPU Time(sec)
V-Cycle	18	1.23×10^{-3}
W-Cycle	20	4.18×10^{-3}
F-Cycle	21	2.24×10^{-2}

Table 3: CPU time(sec) taken for 1 iteration

It can be seen that CPU time taken increases as complex cycle is used eventhough total iteration remained close. This is due to the extra work done at intermediate levels in these complex cycle.

4.5 Optimisation of Multigrid Algorithm

In a V-cycle, one of the way of optimising it is to do more and more iterations on coarse levels as the computational requirement decreases by a factor of $1/4$ as we go deep in level. Theoretically, doing more and more iterations of smoother on coarse level should increase the speed. This speedup was observed by linearly increasing the smoother iteration on intermediate levels. But when the iterations were increased quadratically and even cubically the performance depleted. The best performance was observed with linearly increase at intermediate levels with $\omega = 1$ (19 iterations compared to 40 for 256^2 grid). But in case of W-cycle and F-cycle, there are more parameter that can be changed to optimise the algorithm. One of the key parameter is the structure of these complex cycles. In this work, V-cycle gave the best results compared to W-cycle and F-cycle.

Appendix

Code

```
module global_variables
  implicit none
  integer :: i,j,k,nx,ny,nlev, maxiter, it_smoother, solver, cycle
  integer, parameter :: gs=1, line_sor=2, srj=3, multiG=2
  integer, parameter :: vcycle=1, wcycle=2, fcycle=3
  real :: residual, tol, dx, dy, kw, lx, ly, w
  real :: start_time, end_time
  real, parameter :: pi = 3.1415927
  real, dimension(:,:), allocatable :: u, ukp1, uk,u_init,s
  real, dimension(:), allocatable :: x, y
endmodule

module fnames
  integer, parameter :: input=12, postproc=13, logfile=14
endmodule

program numerical_project

  use global_variables

  call init_simulation()

  residual = 1
  s(:, :) = 0.0
  start_time = 0.0
  end_time = 0.0

  iter = 1
  select case(solver)
  case(gs)
    open(unit=logfile,file='log_gs.dat',status='unknown')
    call cpu_time(start_time)
    write(logfile,*) 'nx = ', nx-1, ' ny = ', ny-1, ' Solver = Gauss Seidel'
    write(logfile,*) 'Iteration      Residual'
    do while(residual>tol .and. iter<maxiter)
      call gauss_seidel(nx,ny,dx,w,s,ukp1)
      call calc_residual(nx,ny,dx,dy,ukp1,residual)
      write(logfile,*) iter,residual
      uk = ukp1
      iter = iter + 1
    end do
    call cpu_time(end_time)
    write(*,*) 'GS taking, Iterations = ', iter, ' with residual = ',residual,' in time = ', end_time-start_time
  case(multiG)
    select case(cycle)
    case(vcycle)
      open(unit=logfile,file='log_mg.dat',status='unknown')
      call init_multigrid(nx,ny,nlev)
      write(logfile,*) 'nx = ', nx-1, ' ny = ', ny-1, ' Solver = Multigrid, Smoother = ', it_smoother
      write(logfile,*) 'Iteration      Residual'
      call cpu_time(start_time)
      do while(residual>tol .and. iter<maxiter)
        call multigrid_VCycle(nx,ny,uk,nlev,it_smoother,w,ukp1)
        call calc_residual(nx,ny,dx,dy,ukp1,residual)
        write(logfile,*) iter,residual
        uk = ukp1
        iter = iter + 1
      end do
      call cpu_time(end_time)
      call end_multigrid()
      write(logfile,*)
      write(*,*) 'Multigrid taking (V-Cycle), Iterations = ', iter, ' with residual = ',residual,' in time = ' &
        , end_time-start_time
    case(wcycle)
      open(unit=logfile,file='log_mg.dat',status='unknown')
      call init_multigrid(nx,ny,nlev)
      write(logfile,*) 'nx = ', nx-1, ' ny = ', ny-1, ' Solver = Multigrid, Smoother = ', it_smoother
      write(logfile,*) 'Iteration      Residual'
      call cpu_time(start_time)
      do while(residual>tol .and. iter<maxiter)
        call multigrid_WCycle(nx,ny,uk,nlev,it_smoother,w,ukp1)
        call calc_residual(nx,ny,dx,dy,ukp1,residual)
        write(logfile,*) iter,residual
        uk = ukp1
```

```

        iter = iter + 1
    end do
    call cpu_time(end_time)
    call end_multigrid()
    write(logfile,*)
    write(*,*) 'Multigrid (W-Cycle) taking, Iterations = ', iter, ' with residual = ',residual,' in time = ' &
        , end_time-start_time

    case(fcycle)
    open(unit=logfile,file='log_mg.dat',status='unknown')
    call init_multigrid(nx,ny,nlev)
    write(logfile,*) 'nx = ', nx-1, ' ny = ', ny-1,' Solver = Multigrid, Smoother = ', it_smoother
    write(logfile,*) 'Iteration      Residual'
    call cpu_time(start_time)
    do while(residual>tol .and. iter<maxiter)
        call multigrid_FCycle(nx,ny,uk,nlev,it_smoother,w,ukp1)
        call calc_residual(nx,ny,dx,dy,ukp1,residual)
        write(logfile,*) iter,residual
        uk = ukp1
        iter = iter + 1
    end do
    call cpu_time(end_time)
    call end_multigrid()
    write(logfile,*)
    write(*,*) 'Multigrid (F-Cycle) taking, Iterations = ', iter, ' with residual = ',residual,' in time = ' &
        , end_time-start_time

    end select
endselect

u(:, :) = ukp1(:, :)

close(logfile)

call write_postproc(nx,ny,x,y,u)
call deallocate_arrays()

end program numerical_project

subroutine init_simulation()
    use global_variables
    call read_input()
    call allocate_arrays()

    do i=1,nx
        x(i) = (i-1)*dx
    enddo

    do i = 1,ny
        y(i) = (i-1)*dy
    enddo

    ! Initialising Domain
    ! call random_generator(nx,ny,-1.0,1.0,u_init)
    ! open(15,file='random_number.dat',status='unknown')
    ! do i=1,ny
    !     write(15,*) u_init(:,i)
    ! enddo
    ! close(15)
    open(17,file='random_number.dat',status='old')
    do i=1,ny
        read(17,*) u_init(:,i)
    enddo
    close(17)

    u_init(:, :) = u_init(:, :)

    ! Setting Boundary Conditions
    u_init(:,1) = sin(kw*x(:))
    u_init(:,ny) = 0
    u_init(1,:) = sin(kw*y(:))
    u_init(nx,:) = 0

    u(:, :) = u_init(:, :)
    ukp1(:, :) = u(:, :)
    uk(:, :) = u(:, :)
endsubroutine init_simulation

subroutine read_input()
    use global_variables
    use fnames

```

```

open(input, file='input.dat', status='old')
read(input,*)
read(input,*)
read(input,*)
read(input,*) nx, ny
read(input,*)
read(input,*) kw
read(input,*)
read(input,*)
read(input,*) solver
read(input,*) it_smoother
read(input,*)
read(input,*) w
read(input,*)
read(input,*) tol, maxiter
read(input,*)
read(input,*) cycle, nlev
nx = nx + 1
ny = ny + 1

dx = 2*pi/(nx-1)
dy = 2*pi/(ny-1)

endsubroutine read_input

subroutine write_postproc(nx,ny,x,y,data)
integer :: i,j,nx,ny
real,dimension(nx,ny),intent(in) :: data
real,dimension(nx), intent(in) :: x
real,dimension(nx), intent(in) :: y

open(12, file='data.dat', status='unknown')
write(12,*) 'TITLE = "Post Processing Tecplot"'
write(12,*) 'VARIABLES = "X", "Y", "U"'
write(12,*) 'ZONE T="BIG ZONE", I=',nx,', J=',ny,', DATAPACKING=POINT'

DO j=1,ny
do i = 1,nx
write(12,*) x(i), y(j), data(i,j)
enddo
enddo
close(12)
endsubroutine

subroutine random_generator(nx,ny,a,b,x)
implicit none
integer, intent(in) :: nx,ny
real,intent(in) :: a,b
real,dimension(nx,ny),intent(out) :: x
real,dimension(nx,ny) :: u
call random_number(u)
x = (b-a)*u + a
end subroutine random_generator

subroutine allocate_arrays()
use global_variables

allocate(u(nx,ny),ukp1(nx,ny),uk(nx,ny),u_init(nx,ny),s(nx,ny))
allocate(x(nx),y(ny))

endsubroutine allocate_arrays

subroutine deallocate_arrays()
use global_variables

deallocate(u,ukp1,uk,u_init,s)
deallocate(x,y)
endsubroutine deallocate_arrays

subroutine calc_residual(nx,ny,dx,dy,data,residual_out)
integer :: i,j
integer, intent(in) :: nx, ny
real, intent(out) :: residual_out
real, intent(in) :: dx, dy
real :: residual
real, dimension(nx,ny), intent(in) :: data

residual = 0

! L1 Norm

```

```

do j=2,ny-1
  do i = 2,nx-1
    residual = residual + abs((1/(dx**2))*(data(i+1,j) -2*data(i,j) + data(i-1,j)) &
      + (1/(dy**2))*(data(i,j+1) -2*data(i,j) + data(i,j-1)))
  enddo
enddo
residual = residual/((nx-2)*(ny-2))

! L2 norm
do j=2,ny-1
  do i = 2,nx-1
    residual = residual + ((1/(dx**2))*(data(i+1,j) -2*data(i,j) + data(i-1,j)) &
      + (1/(dy**2))*(data(i,j+1) -2*data(i,j) + data(i,j-1)))**2
  enddo
enddo

residual = sqrt(residual)/((nx-2)*(ny-2))

residual_out = residual
endsubroutine

module multigrid_mod
  implicit none
  integer :: i,j,n
  integer :: mx1, my1, mx2, my2, smoother ! number of grid points in x and y direction
  real :: dxm, dym, w ! weight for smoother !!! using the same variable again might be a problem !!!
  real, parameter :: pi = 3.141592653589793238462643383279502884197
  real, dimension(:,:), allocatable :: f, gs_src, gsout
  real, dimension(:,:,:), allocatable :: res, err
endmodule

subroutine init_multigrid(nx,ny,nlev)
  use multigrid_mod
  integer, intent(in) :: nx, ny, nlev
  allocate(res(nx,ny,nlev))
  allocate(err(nx,ny,nlev))
  allocate(f(nx,ny))
  allocate(gs_src(nx,ny),gsout(nx,ny))
end subroutine init_multigrid

subroutine multigrid_VCycle(nx,ny,uk,nlev,solver,relax,ukp1)
  use multigrid_mod
  integer, intent(in) :: nx, ny, nlev, solver
  real, intent(in) :: relax
  real, dimension(nx,ny), intent(in) :: uk
  real, dimension(nx,ny), intent(out) :: ukp1
  real, dimension(nx,ny) :: source

  w = relax
  smoother = solver
  mx1 = nx
  my1 = ny
  dxm = 2*pi/(mx1-1)
  dym = 2*pi/(my1-1)

  f(:, :) = uk(:, :)
  source(:, :) = 0.0
  res(:, :, :) = 0.0
  err(:, :, :) = 0.0

  call gauss_seidel(mx1,my1,dxm,w,source,f)

  call calc_r_multigrid()

  ! Restriction
  do n=nlev-1,1
    call fine_to_coarse()
  enddo

  n=nlev
  call fine_to_coarse_coarsest()

  ! Prolongation
  do n=nlev-1,1,-1
    call coarse_to_fine()
  enddo

  f(:, :) = f(:, :) - err(:, :, 1)
  ukp1(:, :) = f(:, :)
endsubroutine multigrid_VCycle

subroutine multigrid_WCycle(nx,ny,uk,nlev,solver,relax,ukp1)

```

```

use multigrid_mod
use ieee_arithmetic
integer, intent(in) :: nx, ny, nlev, solver
real, intent(in) :: relax
real, dimension(nx,ny), intent(in) :: uk
real, dimension(nx,ny), intent(out) :: ukp1
real, dimension(nx,ny) :: source

w = relax
smoother = solver
mx1 = nx
my1 = ny
dxm = 2*pi/(mx1-1)
dym = 2*pi/(my1-1)

f(:, :) = uk(:, :)
source(:, :) = 0.0
res(:, :, :) = 0.0
err(:, :, :) = 0.0

call gauss_seidel(mx1, my1, dxm, w, source, f)

call calc_r_multigrid()

! Restriction
do n=2, nlev-1
    call fine_to_coarse()
enddo

n = nlev
call fine_to_coarse_coarsest()

do n = nlev-1, nlev-2, -1
    call coarse_to_fine()
enddo

do n = nlev-1, nlev-1
    call fine_to_coarse()
enddo

n = nlev
call fine_to_coarse_coarsest()

do n=nlev-1, 1, -1
    call coarse_to_fine()
enddo

f(:, :) = f(:, :) - err(:, :, 1)
ukp1(:, :) = f(:, :)

end subroutine multigrid_WCycle

subroutine multigrid_FCycle(nx, ny, uk, nlev, solver, relax, ukp1)
use multigrid_mod
integer, intent(in) :: nx, ny, nlev, solver
real, intent(in) :: relax
real, dimension(nx,ny), intent(in) :: uk
real, dimension(nx,ny), intent(out) :: ukp1
real, dimension(nx,ny) :: source

w = relax
smoother = solver
mx1 = nx
my1 = ny
dxm = 2*pi/(mx1-1)
dym = 2*pi/(my1-1)

f(:, :) = uk(:, :)
source(:, :) = 0.0
res(:, :, :) = 0.0
err(:, :, :) = 0.0

call gauss_seidel(mx1, my1, dxm, w, source, f)

call calc_r_multigrid()

! Restriction

```

```

do n=2,nlev-1
    call fine_to_coarse()
enddo

n = nlev
call fine_to_coarse_coarsest()

do n = nlev-1, nlev-2, -1
    call coarse_to_fine()
enddo

do n = nlev-1, nlev-1
    call fine_to_coarse()
enddo

n = nlev
call fine_to_coarse_coarsest()

do n=nlev-1,nlev-3,-1
    call coarse_to_fine()
enddo

do n=nlev-2,nlev-1
    call fine_to_coarse()
enddo

n = nlev
call fine_to_coarse_coarsest()

do n=nlev-1,1,-1
    call coarse_to_fine()
enddo

f(:, :) = f(:, :) - err(:, :, 1)
ukp1(:, :) = f(:, :)
end subroutine multigrid_FCycle

subroutine fine_to_coarse()
    use multigrid_mod

    mx2 = (mx1-1)/2 + 1
    my2 = (my1-1)/2 + 1
    dxm = 2*pi/(mx2-1)
    dym = 2*pi/(my2-1)
    call restriction()

    gsout(:, :) = 0.0
    gs_src(:, :) = 0.0
    gs_src(1:mx2, 1:my2) = res(1:mx2, 1:my2, n)
    do k = 1, n
        call gauss_seidel(mx2, my2, dxm, w, gs_src(1:mx2, 1:my2), gsout(1:mx2, 1:my2))
    end do
    err(1:mx2, 1:my2, n) = gsout(:, :)
    call calc_r_error(mx2, my2)

    mx1 = mx2
    my1 = my2
endsubroutine fine_to_coarse

subroutine fine_to_coarse_coarsest()
    use multigrid_mod

    mx2 = (mx1-1)/2 + 1
    my2 = (my1-1)/2 + 1
    dxm = 2*pi/(mx2-1)
    dym = 2*pi/(my2-1)
    call restriction()

    gsout(:, :) = 0.0
    gs_src(:, :) = 0.0
    gs_src(1:mx2, 1:my2) = res(1:mx2, 1:my2, n)
    do k = 1, 1
        call gauss_seidel(mx2, my2, dxm, w, gs_src(1:mx2, 1:my2), gsout(1:mx2, 1:my2))
    end do
    err(1:mx2, 1:my2, n) = gsout(:, :)
    call calc_r_error(mx2, my2)

    mx1 = mx2
    my1 = my2
endsubroutine fine_to_coarse_coarsest

```

```

subroutine coarse_to_fine()
  use multigrid_mod
  mx1 = (mx2-1)*2 + 1
  my1 = (my2-1)*2 + 1
  dxm = 2*pi/(mx1-1)
  dym = 2*pi/(my1-1)

  call prolongation()

  gsout(:, :) = err(1:mx1, 1:my1, n)
  gs_src(:, :) = 0.0
  gs_src(1:mx1, 1:my1) = res(1:mx1, 1:my1, n)
  do k = 1, 1
    call gauss_seidel(mx1, my1, dxm, w, gs_src(1:mx1, 1:my1), gsout(1:mx1, 1:my1))
  end do
  err(1:mx1, 1:my1, n) = gsout(:, :)

  mx2 = mx1
  my2 = my1

endsubroutine coarse_to_fine

subroutine restriction()
  use multigrid_mod
  integer :: i1, j1, incx, incy

  incx = 0
  incy = 0
  do j = 2, my2 - 1
    do i = 2, mx2 - 1
      i1 = 2*i - 1
      j1 = 2*j - 1
      res(i, j, n) = (res(i1, j1, n-1) &
        + 0.5*(res(i1-1, j1, n-1) + res(i1+1, j1, n-1) + res(i1, j1+1, n-1) + res(i1, j1-1, n-1)) &
        + 0.25*(res(i1-1, j1-1, n-1) + res(i1-1, j1+1, n-1) + res(i1+1, j1-1, n-1) + res(i1+1, j1+1, n-1)))/(4)
    enddo
  enddo
endsubroutine restriction

subroutine prolongation()
  use multigrid_mod
  integer :: i1, j1

  j1 = 1
  do j=2, mx1-1, 2
    j1 = j1 + 1
    i1 = 1
    do i=2, mx1-1, 2
      i1 = i1 + 1
      !element in fine is related to top right coarse element
      ! n+1 because value of n is interpolated from level (n+1) (eg level 1 obtained from level 2)
      err(i, j, n) = err(i, j, n) + (err(i1, j1, n+1) + err(i1-1, j1, n+1) + err(i1, j1-1, n+1) + err(i1-1, j1-1, n+1))/4
    enddo
  enddo

  j1 = 0
  do j=2, my1-1, 2
    j1 = j1 + 1
    i1 = 1
    do i=3, mx1-2, 2
      i1 = i1 + 1
      !element in fine is related to top coarse element
      err(i, j, n) = err(i, j, n) + (err(i1, j1, n+1) + err(i1, j1+1, n+1))/2
    enddo
  enddo

  j1 = 1
  do j=3, my1-2, 2
    j1 = j1 + 1
    i1 = 0
    do i=2, mx1-1, 2
      i1 = i1 + 1
      !element in fine is related to left coarse element
      err(i, j, n) = err(i, j, n) + (err(i1, j1, n+1) + err(i1+1, j1, n+1))/2
    enddo
  enddo

  do j=3, my1-2, 2
    do i=3, mx1-2, 2
      i1 = (i+1)/2
      j1 = (j+1)/2
    enddo
  enddo

```

```

        err(i,j,n) = err(i,j,n) + err(i1,j1,n+1)
    enddo
enddo

endsubroutine prolongation

subroutine gauss_seidel(nx,ny,del,w,source,data)
    integer :: i,j
    integer, intent(in) :: nx,ny
    real, intent(in) :: del,w
    real, dimension(nx,ny), intent(inout) :: data
    real, dimension(nx,ny), intent(in) :: source
    real, dimension(nx,ny) :: data_old

    data_old(:, :) = data(:, :)

    do j = 2, ny-1
        do i = 2, nx-1
            data(i,j) = 0.25*(data(i+1,j)+data(i-1,j)+data(i,j+1)+data(i,j-1)) - (del**2)*source(i,j)/4
        end do
    end do

    data(:, :) = (1-w)*data_old(:, :) + w*data(:, :)

endsubroutine gauss_seidel

subroutine calc_r_multigrid()
    use multigrid_mod
    res = 0

    do j=2,my1-1
        do i = 2,mx1-1
            res(i,j,1) = ((1/(dxm**2))*(f(i+1,j) -2*f(i,j) + f(i-1,j)) &
                + (1/(dym**2))*(f(i,j+1) -2*f(i,j) + f(i,j-1)))
        enddo
    enddo
endsubroutine

subroutine calc_r_error(nxr,nyr)
    use multigrid_mod
    integer, intent(in) :: nxr, nyr

    do j=2,nxr-1
        do i = 2,nyr-1
            res(i,j,n) = -(((1/(dxm**2))*(err(i+1,j,n) -2*err(i,j,n) + err(i-1,j,n)) &
                + (1/(dym**2))*(err(i,j+1,n) -2*err(i,j,n) + err(i,j-1,n))) - res(i,j,n))
        enddo
    enddo
endsubroutine

subroutine end_multigrid()
    use multigrid_mod
    deallocate(res)
    deallocate(err)
    deallocate(f)
    deallocate(gs_src)
    deallocate(gsout)
end subroutine end_multigrid

```

Input

```

|===== Input File for Numerical Methods Project =====|
|----- Simulation Parameters -----|
nx ny (internal nodes only, +2 added in code for boundaries, +1 for fc arrangement)
256 256
wavenumber for BC
4
|----- Solver Settings -----|
Solver(1. Gauss Seidel 2.Multigrid)
2
Smoother(1. Gauss Seidel 2.Line SOR 3.SRJ)
1
Relaxation
1.00
Maximum Tolerance    Maximum Iteration
1E-6                 20000
Cycle(1. Vcycle, 2. Wcycle, 3.Fcycle)    Number of Levels
1                                         8

```